Solving Differential Equations using Physics Informed Neural Networks

Rohan Agrawal

B.E. in Mathematics and Computing
BITS Pilani, K.K. Birla Goa Campus

2nd May, 2026

A report submitted as part of Study Project MAC F266
under guidance of Prof. P. Danumjaya

Abstract

Physics-Informed Neural Networks (PINNs) are a class of machine-learning models that embed physical laws—expressed as ordinary or partial differential equations—directly into the training objective of a neural network. This report provides a self-contained treatment of PINNs for solving ordinary differential equations (ODEs), beginning with the mathematical foundations of feedforward neural networks and progressing through three distinct constraint-enforcement strategies for Initial Value Problems: soft embedding, hard (trial-solution) embedding, and an integral/quadrature formulation.

The report then extends the treatment to Boundary Value Problems (BVPs), covering soft and hard boundary-condition enforcement with detailed numerical experiments. Advanced topics include systems of ODEs (illustrated with the Lotka–Volterra model), a hybrid Euler–neural-network method, inverse-problem parameter estimation from sparse and noisy observations, the challenges of high-frequency solutions and spectral bias, and the Finite Basis PINN (FBPINN) and Extreme Learning Machine FBPINN (ELM-FBPINN) architectures that address these limitations.

Practical comparisons of PyTorch and JAX implementations highlight speed advantages of up to 7× for JAX, while ELM-FBPINNs reduce training times by several orders of magnitude by replacing gradient descent with a direct linear-algebra solve.

Keywords: Physics-Informed Neural Networks, Ordinary Differential Equations, Boundary Value Problems, Collocation Methods, Automatic Differentiation, Inverse Problems, FBPINNs, ELM, JAX, PyTorch.

Contents

1 Neural Networks Background

1.1 Mathematical Definition of a Feedforward Neural Network

A feedforward neural network (FNN) defines a parametric map

$$f_\theta : \mathbb{R}^n \to \mathbb{R}^m.$$

Let $a^{(0)} = x \in \mathbb{R}^n$. For layers $l = 1, \dots, L$ the forward pass is defined by the recurrence

$$z^{(l)} = W^{(l)} a^{(l-1)} + b^{(l)}, \qquad W^{(l)} \in \mathbb{R}^{d_l \times d_{l-1}},\ b^{(l)} \in \mathbb{R}^{d_l},$$

$$a^{(l)} = \sigma^{(l)}\left(z^{(l)}\right),$$

where $\sigma^{(l)}$ is a (possibly layer-dependent) nonlinear activation function. The network output is $f_\theta(x) = a^{(L)}$.

1.2 Functional (Composition) Viewpoint

Define affine maps $A^{(l)}(u) = W^{(l+1)}u + b^{(l+1)}$ for $l = 0, \dots, L-1$. Then the network can be written compactly as

$$f_\theta = \sigma^{(L)} \circ A^{(L-1)} \circ \cdots \circ \sigma^{(1)} \circ A^{(0)}.$$

The trainable parameters are

$$\theta = \left\{(W^{(l)}, b^{(l)})\right\}_{l=1}^{L}.$$

The network is a *finite-dimensional family of nonlinear maps*; its expressive power comes from the composition of affine maps with nonlinearities.

1.3 Neuron Model

A single neuron computes

$$z = \langle w, x \rangle + b, \qquad a = \sigma(z),$$

where $\langle w, x \rangle$ is the inner product (projection of x onto the weight vector w), and σ introduces nonlinearity. Without σ the entire network collapses to a single affine map regardless of depth.

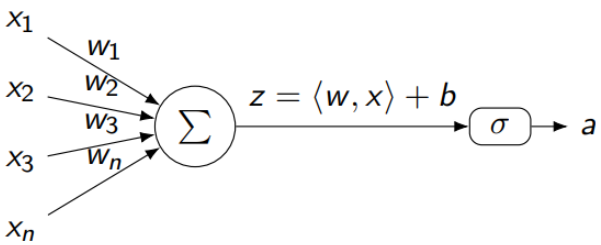


Figure 1: Neuron model: weighted sum followed by a nonlinear activation.

Depth and Width.

- **Depth**: the total number of layers (hidden + output).
- **Width**: the number of neurons per layer (may vary across layers).

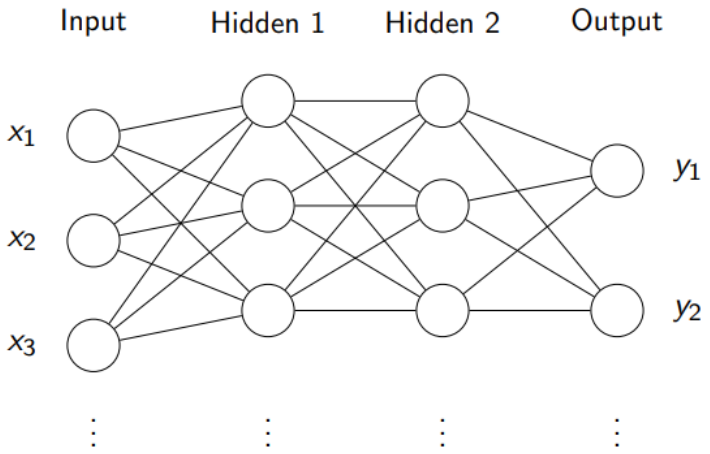


Figure 2: A multilayer feedforward network. Adjacent layers are fully connected and information flows strictly forward.

1.4 Activation Functions in Physics-Based Learning

Physics-based models require computing Jacobians and Hessians of the network output with respect to its inputs (to evaluate differential-equation residuals), so **smooth activation functions** are essential.

- $\tanh(x)$: smooth, bounded, odd-symmetric—useful for signed outputs.
- $\text{Softplus}(x) = \log(1 + e^x)$: smooth surrogate for ReLU.
- $\text{Swish}(x) = x \cdot \sigma(x)$: smooth, non-monotone, strong gradient flow.

Why avoid ReLU in PINNs? ReLU is not differentiable at $x = 0$, so its second (and higher) derivatives are zero almost everywhere. This makes it unsuitable for loss terms that involve second-order derivatives of the network.

1.5 Research Questions on Activation Functions

Given data $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N$, several open questions arise:

- How do we choose an *effective* activation function?
- Can the activation function itself be *learned* as part of training?
- How does performance vary with network depth, and should the activation function change across layers?
- Is a single activation sufficient, or should it vary with depth?

1.6 Universal Approximation

Universal Approximation Theorem (Cybenko, 1989). If σ is a continuous sigmoidal activation, then for any continuous f on a compact set $K \subset \mathbb{R}^n$ and any $\varepsilon > 0$, there exists a single-hidden-layer network f_θ such that

$$\|f - f_\theta\|_\infty < \varepsilon.$$

Key caveats: the theorem is an existence result and says nothing about optimisation or generalisation; depth improves parameter efficiency for structured functions; and the required network width is not specified.

2 Introduction to Physics-Informed Neural Networks

2.1 Motivation

Physics-Informed Neural Networks (PINNs) incorporate **physical laws (ODEs/PDEs)** directly into the loss function used during training. They are particularly valuable when:

- Measurement data are scarce.
- The governing equations of the system are known.

PINNs unify the solution of two classes of problems:

- **Forward problems**: given the governing equations and parameters, find the solution.
- **Inverse problems**: given data, infer unknown parameters.

2.2 Forward vs. Inverse Problems

Forward Problem.

$$\frac{du}{dt} = f(t, u), \qquad u(t_0) = u_0. \qquad \text{Find } u(t).$$

Inverse Problem. Given observed data for $u(t)$, estimate unknown parameters λ in

$$\frac{du}{dt} = f(t, u; \lambda).$$

The loss functions differ:

$$\mathcal{L}_{\text{forward}} = \mathcal{L}_{\text{physics}} + \mathcal{L}_{\text{BC}}, \qquad \mathcal{L}_{\text{inverse}} = \mathcal{L}_{\text{data}} + \mathcal{L}_{\text{physics}} + \mathcal{L}_{\text{BC}}.$$

3 PINN Methods for First-Order ODEs (IVPs)

3.1 Problem Setup

$$\frac{du}{dt} = F(t, u(t)), \qquad u(0) = u_0, \qquad t \in [0, 1].$$

The solution $u(t)$ is approximated by a neural network $u_\theta(t)$, trained by minimising the ODE residual at collocation points $\{t_i\}$ (no labelled data required).

3.2 Method 1: Soft Initial-Condition Embedding

Ansatz: $u_\theta(t) = f_\theta(t)$.

$$\mathcal{L}_{IC} = |f_\theta(0) - u_0|^2, \qquad \mathcal{L}_{eq} = \frac{1}{N}\sum_{i=1}^{N} \left|f'_\theta(t_i) - F(t_i, f_\theta(t_i))\right|^2, \qquad \mathcal{L} = \mathcal{L}_{IC} + \lambda\, \mathcal{L}_{eq}.$$

3.3 Method 2: Hard Initial-Condition Embedding

Trial solution: $u_\theta(t) = u_0 + t\, f_\theta(t)$. Automatically satisfies $u_\theta(0) = u_0$, so only the ODE residual is minimised:

$$R(t) = f_\theta(t) + t\, f'_\theta(t) - F(t,\, u_0 + t\, f_\theta(t)), \qquad \mathcal{L} = \frac{1}{N}\sum_{i=1}^{N} |R(t_i)|^2.$$

3.4 Method 3: Integral (Quadrature) Formulation

$$u(t) = u_0 + \int_0^t F(s, u(s))\, ds.$$

Approximating the integral numerically:

$$\mathcal{L} = \frac{1}{N}\sum_{i=1}^{N} \left[u_\theta(t_i) - u_0 - \sum_{j=1}^{M} F(s_{ij}, u_\theta(s_{ij}))\, w_j \right]^2.$$

- $\{t_i\}$: collocation points (where the differential equation is enforced)
- $\{s_{ij}\}$: quadrature nodes (used to approximate the integral)
- $\{w_j\}$: weights (determined by the chosen quadrature rule)

4 PINN Methods for Second-Order ODEs (IVPs)

4.1 Problem Setup

$$u''(t) = F(t, u, u'), \qquad u(0) = u_0, \quad u'(0) = v_0.$$

4.2 Method 1: Soft Constraints

$$\mathcal{L}_{IC} = |f_\theta(0) - u_0|^2 + |f'_\theta(0) - v_0|^2, \qquad \mathcal{L}_{eq} = \frac{1}{N}\sum_{i=1}^{N} |f''_\theta(t_i) - F(t_i, f_\theta(t_i), f'_\theta(t_i))|^2 .$$

$$\mathcal{L}(\theta) = \mathcal{L}_{IC}(\theta) + \lambda\, \mathcal{L}_{eq}(\theta).$$

4.3 Method 2: Hard-Embedding Initial Conditions

Trial solution: $u_\theta(t) = u_0 + v_0\, t + t^2\, f_\theta(t)$, satisfying both ICs exactly. Derivatives:

$$u'_\theta = v_0 + 2t\, f_\theta + t^2\, f'_\theta, \qquad u''_\theta = 2\, f_\theta + 4t\, f'_\theta + t^2\, f''_\theta.$$

$$\mathcal{L}(\theta) = \frac{1}{N}\sum_{i=1}^{N} |u''_\theta(t_i) - F(t_i, u_\theta(t_i), u'_\theta(t_i))|^2.$$

4.4 Reduction to a First-Order System

Introduce $y_1 = u$, $y_2 = u'$:

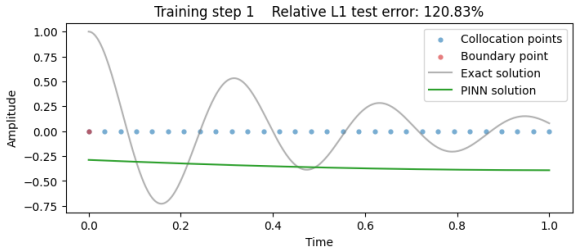
$$y'_1 = y_2, \qquad y'_2 = F(t, y_1, y_2), \qquad y_1(0) = u_0,\ y_2(0) = v_0.$$

Standard first-order PINN methods then apply directly.

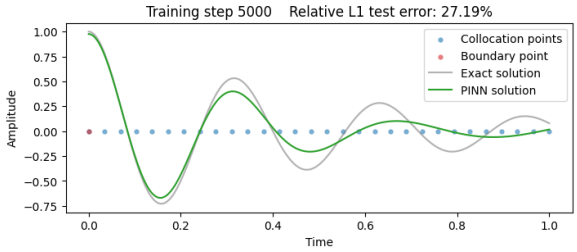
5 IVP Implementation and Results

5.1 Case Study 1: Damped Harmonic Oscillator (Soft IC)

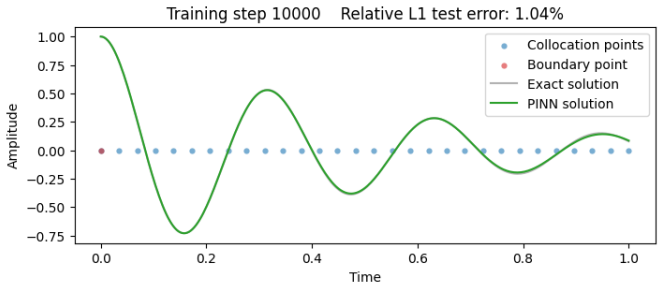
Problem: $mu'' + \mu u' + ku = 0$, $u(0) = 1$, $u'(0) = 0$. Under-damped: $\delta = \mu/(2m) < \omega_0 = \sqrt{k/m}$. Exact: $u(t) = e^{-\delta t}(2A\cos(\phi + \omega t))$.



(a) Training step 1.



(b) Training step 5,000.



(c) Training step 10,000 (PyTorch, ≈ 75 s).

Figure 3: Training progression: damped harmonic oscillator, soft IC.

5.2 Case Study 2: Damped Harmonic Oscillator (JAX)

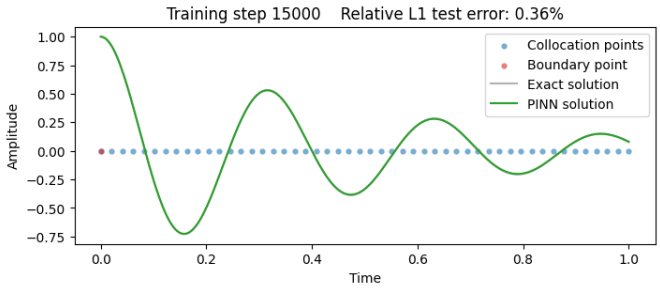
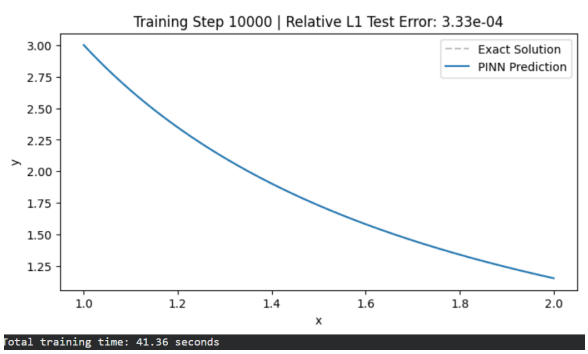


Figure 4: JAX result at step 15,000. Training time: $\approx$ 13 s—roughly 6$\times$ faster than PyTorch at comparable accuracy.

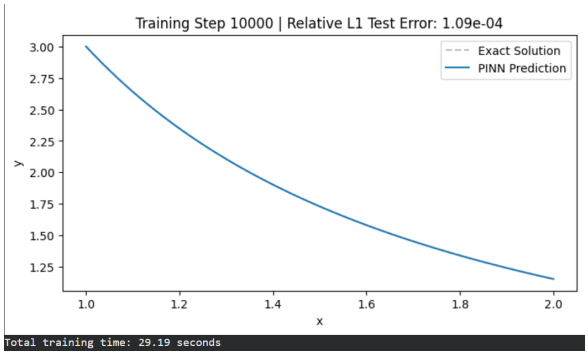
JAX advantages: JIT compilation eliminates Python overhead; `vmap` vectorises residual evaluation; higher-order automatic differentiation is seamless.

5.3 Case Study 3: Cauchy–Euler Equation

Problem: $x^2y'' + 3xy' + y = 0$, $x \in [1, 2]$, $y(1) = 3$, $y'(1) = -4$. Exact: $y(x) = (3 - \log x)/x$.



(a) Soft IC result (PyTorch, ≈ 41 s).



(b) Hard IC result (PyTorch, ≈ 29 s).

Figure 5: Cauchy–Euler equation: soft vs. hard IC embedding. Hard IC achieves faster and more stable convergence.

5.4 Case Study 4: Quadrature Loss / First-Order System

Problem: $d^2y/dx^2 = 2x$, $y(0) = 0$. Exact: $y(x) = x^3/3$.

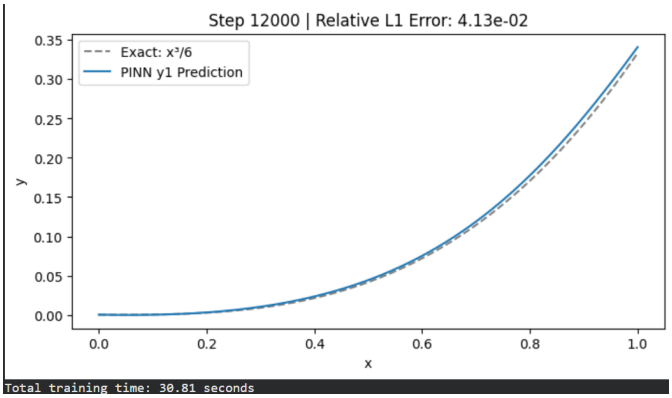


Figure 6: Quadrature loss result (PyTorch, ≈ 31 s). Small residual error is attributed to trapezoidal integration.

5.5 Case Study 5: Lotka–Volterra System

$$\frac{du}{dt} = (2 - v)\,u, \quad \frac{dv}{dt} = (u - 1)\,v, \quad t \in [0, 1],\ u(0) = 3,\ v(0) = 2.$$

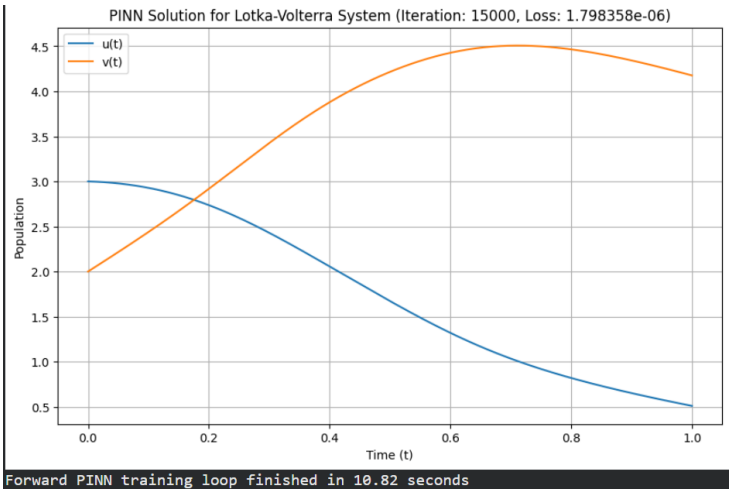
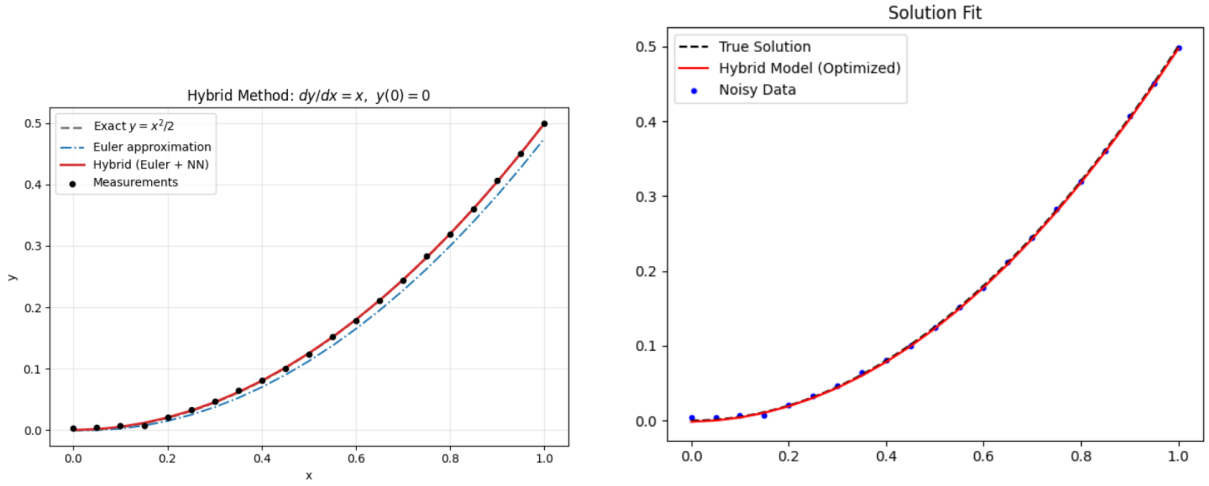


Figure 7: Lotka–Volterra predator-prey system (JAX, ≈ 11 s): characteristic oscillatory dynamics recovered with high accuracy.

5.6 Case Study 6: Hybrid Euler + NN Method

Problem: $dy/dx = x$, $y(0) = 0$. Exact: $y(x) = x^2/2$.

The Euler approximation is corrected by a learned network: $y(x) = y_{Eu}(x) + h_\theta(x)$. Achieved error: $\sim 10^{-5}$.



(a) Solution comparison. (b) Error plot.

Figure 8: Hybrid Euler + NN: very high precision ($\sim 10^{-5}$) with minimal data.

5.7 Case Study 7: Inverse Problem — Estimating α

Problem: $dy/dx = \alpha x$, $y(0) = 0$. True $\alpha = 1$.

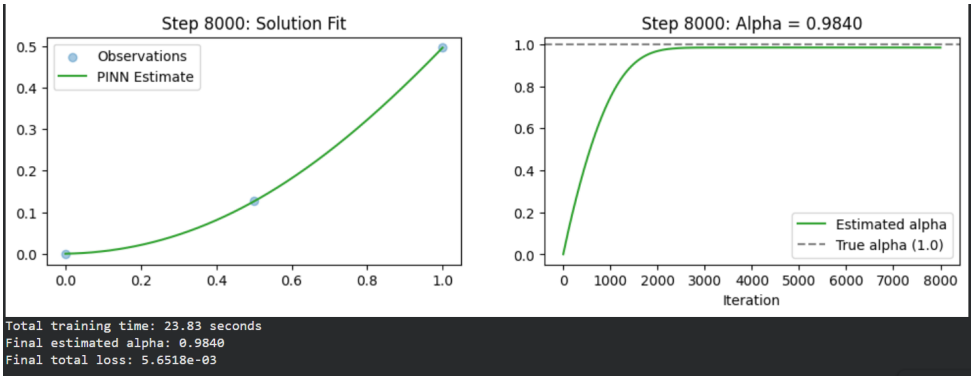


Figure 9: Inverse problem: estimated $\hat{\alpha} \approx 0.984$ with 3 data points.

No. of Data Points	**Estimated $\hat{\alpha}$**	**Absolute Error**
11	1.0131	0.0131
6	1.0155	0.0155
3	0.9840	0.0160

6 Boundary Value Problems (BVPs)

6.1 Standard Mathematical Formulation

We seek $y(t)$ on $t \in [a, b]$ satisfying

$$\frac{d^2y}{dt^2} = f(t, y, y'), \qquad t \in (a, b),$$

with boundary conditions at *both* ends:

$$y(a) = \alpha, \qquad y(b) = \beta.$$

Unlike IVPs, the solution is anchored at two separate points, which changes how constraints are built into the PINN loss.

6.2 Method 1: Soft Enforcement of Boundary Conditions

Ansatz: $y(t) = f_\theta(t)$.

$$\mathcal{L}_{BC}(\theta) = |f_\theta(a) - \alpha|^2 + |f_\theta(b) - \beta|^2, \qquad \mathcal{L}_{eq}(\theta) = \frac{1}{N}\sum_{i=1}^{N} |f''_\theta(t_i) - f(t_i, f_\theta(t_i))|^2 .$$

$$\mathcal{L}(\theta) = \mathcal{L}_{BC}(\theta) + \lambda\, \mathcal{L}_{eq}(\theta), \qquad \lambda > 0.$$

6.2.1 Model Problem

$$-\frac{d^2u}{dx^2} + \pi^2 u - 2\pi^2 \sin(\pi x) = 0, \quad x \in [0, 1], \qquad u(0) = 0,\ u(1) = 0.$$

Exact solution: $u(x) = \sin(\pi x)$. Network: 1 input; 1 hidden layer (32 neurons); 1 output; 30 collocation points; 15,000 epochs (Adam).

Loss:

$$\mathcal{L}(\theta) = \underbrace{|u_\theta(0)|^2 + |u_\theta(1)|^2}_{\mathcal{L}_{BC}} + \lambda \underbrace{\frac{1}{N}\sum_{i=1}^{N} \left| -u''_\theta(x_i) + \pi^2 u_\theta(x_i) - 2\pi^2 \sin(\pi x_i) \right|^2}_{\mathcal{L}_{eq}} .$$

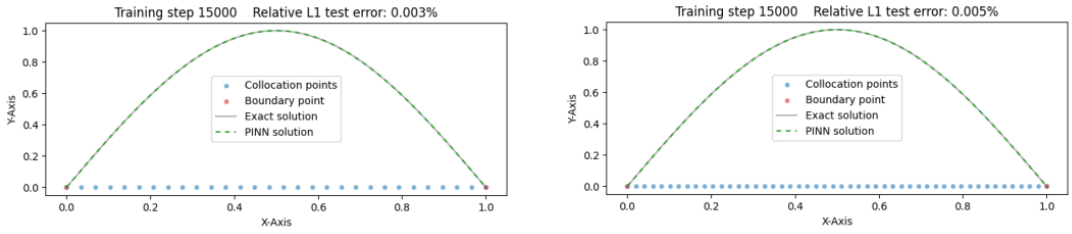


Figure 10: BVP soft-constraint forward problem (step 15,000). **Left**: PyTorch — 53.67 s, relative error 0.003%. **Right**: JAX — 8.27 s, relative error 0.005% ($\approx 7\times$ faster).

6.3 Method 2: Hard Embedding of Boundary Conditions

6.3.1 General Ansatz

For $u(0) = \alpha$, $u(1) = \beta$:

$$u_\theta(t) = (1-t)\,\alpha + t\,\beta + t(1-t)\,f_\theta(t).$$

One verifies $u_\theta(0) = \alpha$ and $u_\theta(1) = \beta$ exactly, so $\mathcal{L}_{BC}$ drops out and

$$\mathcal{L}(\theta) = \frac{1}{N}\sum_{i=1}^{N} |R(t_i;\theta)|^2.$$

6.3.2 Application to the Model Problem

With $\alpha = \beta = 0$, the ansatz reduces to

$$u_\theta(x) = x(1-x)\cdot f_\theta(x).$$

The network is mathematically incapable of violating the boundary conditions, allowing the optimiser to focus entirely on the ODE residual.

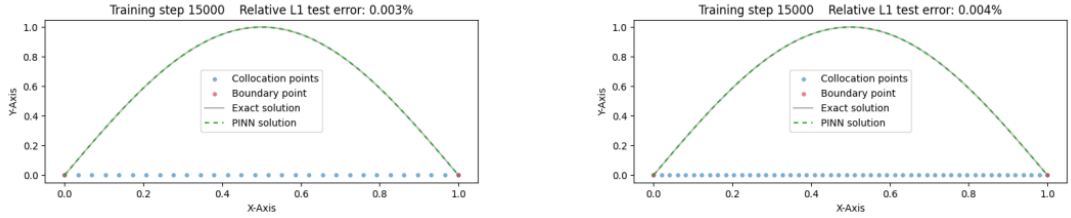


Figure 11: BVP hard-embedding forward problem (step 15,000). **Left**: PyTorch — 51.88 s, relative error 0.003%. **Right**: JAX — 5.45 s, relative error 0.004%.

6.4 BVP Inverse Problem: Parameter Identification

The parameter π is replaced by an unknown A:

$$-\frac{d^2u}{dx^2} + A^2u - 2A^2\sin(Ax) = 0, \quad x \in [0,1], \qquad u(0) = 0,\ u(1) = 0.$$

The network simultaneously approximates $u(x)$ and identifies $A \approx \pi$ from N noisy measurements $\{x_j, \hat{u}_j\}$.

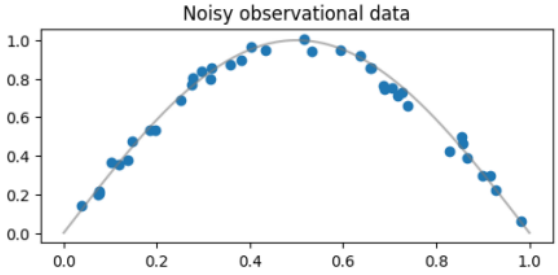


Figure 12: Noisy observational data for the BVP inverse problem.

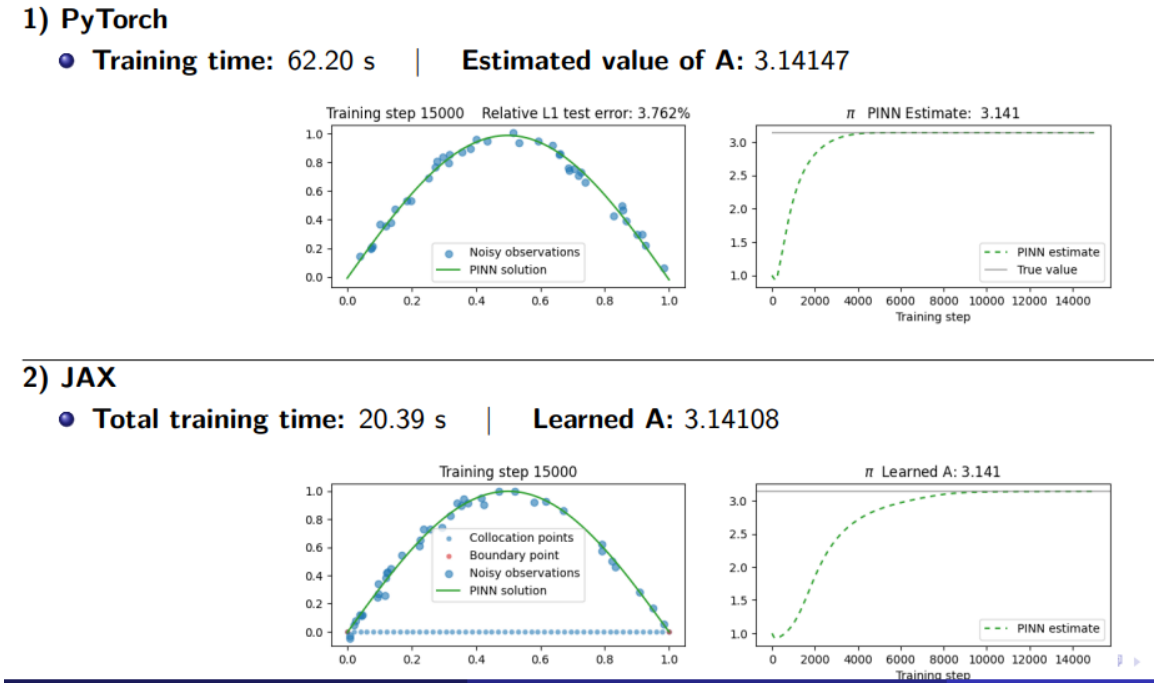


Figure 13: BVP inverse problem results. **PyTorch**: 62.20 s, estimated $A = 3.14147$. **JAX**: 20.39 s, learned $A = 3.14108$. Both converge to π with high precision from noisy data.

7 Challenges: High Frequency and Spectral Bias

Standard PINNs struggle with high-frequency or multi-scale problems. For a damped harmonic oscillator, as frequency ω increases:

- Required collocation points grow as $\propto \omega^d$.
- Larger networks are needed, scaling as $\propto f(\omega)$.
- **Spectral bias**: neural networks inherently learn low-frequency components first, significantly slowing convergence for high-frequency solutions.

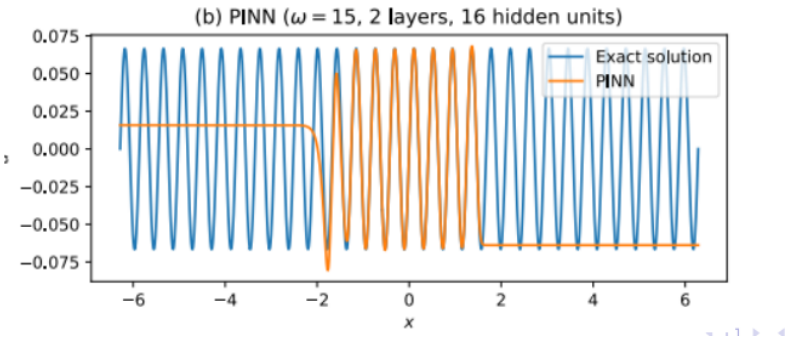


Figure 14: Spectral bias: a standard PINN with $\omega = 15$, 2 layers, 16 hidden units fails to capture the high-frequency oscillations of the exact solution.

8 Finite Basis PINNs (FBPINNs)

8.1 Divide-and-Conquer Strategy

FBPINNs [**?**] address spectral bias by dividing the domain Ω into many smaller, overlapping subdomains and fitting a separate small neural network NN_j in each.

8.1.1 Global Solution Ansatz

$$\hat{u}(x,\theta) = \sum_{j=1}^{J} w_j(x) \cdot \text{unnorm} \circ NN_j \circ \text{norm}_j(x).$$

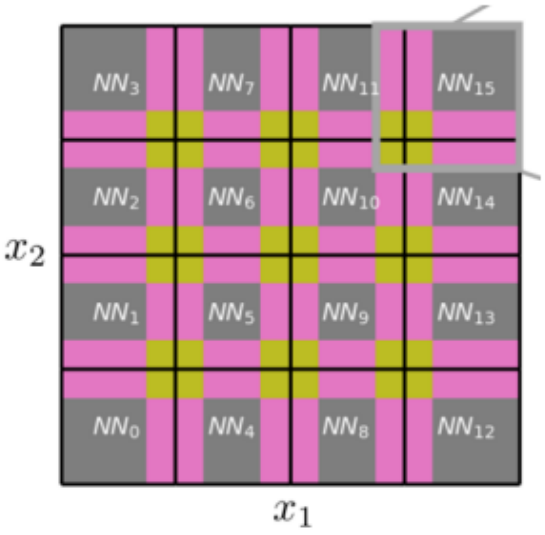


Figure 15: FBPINN domain decomposition into overlapping subdomains, each handled by a local network NN_j.

8.2 Window Functions and Normalisation

- **Window functions** w_j: globally defined; weight each subdomain's contribution; ensure smooth transitions between overlapping models.
- **Subdomain normalisation** norm_j: maps input x to $[-1, 1]$ locally, making learning easier for each NN_j.
- **Output unnormalisation**: scales the local output back to the global physical domain.

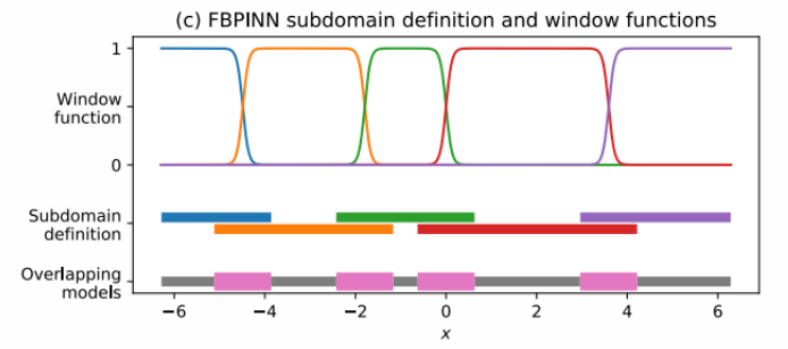


Figure 16: FBPINN subdomain definition and window functions. Coloured curves are the window functions $w_j(x)$; horizontal bars show the subdomain extents.

8.3 Training Loss

Despite the multi-network architecture, FBPINNs use the same loss as standard PINNs:

$$\mathcal{L}(\theta) = \frac{1}{N}\sum_{i=1}^{N} |\mathcal{N}[\hat{u}(t_i, \theta)] - f(t_i)|^2.$$

8.4 Implementation Example

Problem:

$$\frac{dy}{dx} - 2y - x = 0, \qquad y(0) = 1. \qquad \text{Exact: } y(x) = \frac{5e^{2x} - 2x - 1}{4}.$$

Domain decomposition ($J = 4$, $x \in [0, 1]$):

Subdomain	**Interval**
Ω_1	$[0.00,\ 0.35]$
Ω_2	$[0.25,\ 0.60]$
Ω_3	$[0.50,\ 0.85]$
Ω_4	$[0.75,\ 1.00]$

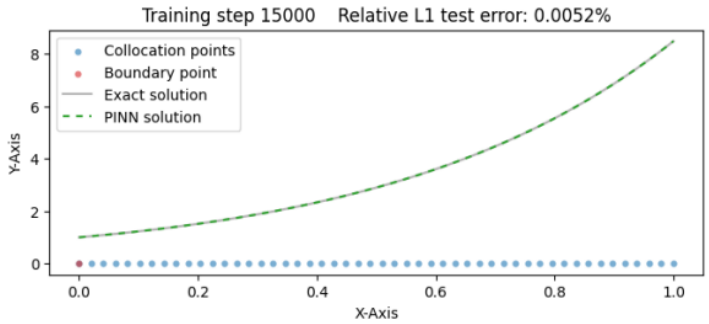


Figure 17: FBPINN approximation at step 15,000. Training time: 13.02 s; relative error: 0.01%.

9 ELM-FBPINNs: Turning Optimisation into Linear Algebra

9.1 Motivation

Standard FBPINNs still rely on gradient descent, which is slow for non-convex loss functions. The computational overhead of backpropagation remains a limiting factor even with JAX.

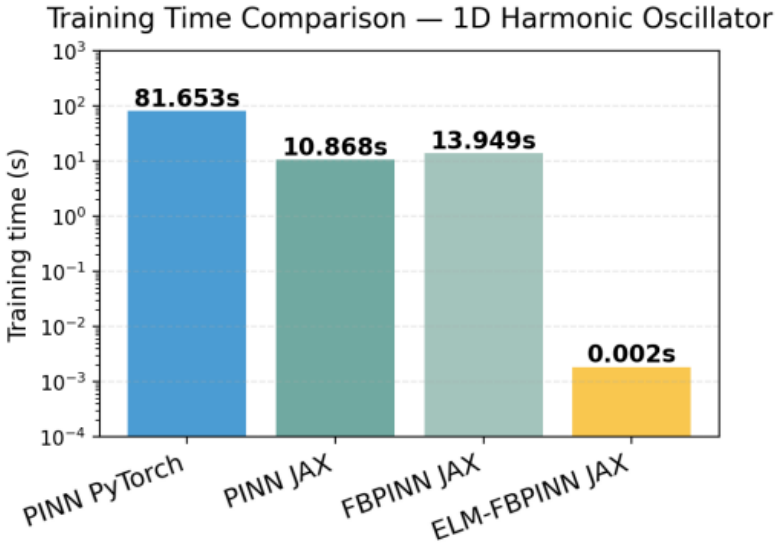


Figure 18: Training time comparison on the 1D harmonic oscillator.

9.2 The Extreme Learning Machine (ELM) Approach

Within each subdomain, hidden weights are randomly initialised and **fixed**; only the output layer is trained. This transforms the loss into a **convex quadratic** problem, solvable directly via normal equations:

- **Fixed basis**: random hidden weights, only output layer trained.
- **Convexity**: least-squares problem with a global minimum.
- **Linear algebra**: solved via normal equations—no gradient descent.

The result is a training-time reduction of **four orders of magnitude** relative to PyTorch PINNs, while maintaining comparable accuracy.

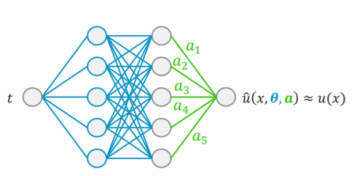


Figure 19: ELM architecture used within each subdomain. Hidden layer weights are randomly initialised and fixed, while only the output layer is trained, reducing the learning problem to a linear least-squares system.

10 Comparison of Methods

Table 1: Summary of all PINN variants covered in this report.

Method	Advantages	Limitations
Soft Embedding (IVP/BVP)	Simple; fully general framework.	Requires tuning λ; may converge slowly.
Hard Embedding (IVP/BVP)	Satisfies IC/BC exactly; faster, more stable.	Needs a problem-specific trial function.
Quadrature Loss	Integral consistency; stable for some problems.	Expensive; not applicable to BVPs.
Hybrid Euler+NN	Very high accuracy ($\sim 10^{-5}$).	Requires initial Euler solution.
FBPINN	Overcomes spectral bias; parallelisable.	Still uses gradient descent.
ELM-FBPINN	Thousands of times faster; convex solve.	Fixed hidden weights limit subdomain expressivity.

11 Discussion and Future Prospects

The methods presented form a progression from vanilla PINNs to highly efficient architectures. Several directions remain open:

- **Practical applications**: applying PINNs and FBPINNs to real-world physical systems (fluid dynamics, heat transfer, structural mechanics).
- **Optimiser robustness**: exploring second-order optimisers (L-BFGS), adaptive collocation, and curriculum-learning strategies.
- **Deeper ELM-FBPINN theory**: approximation guarantees and generalisation bounds for the fixed-basis approach.
- **Extension to PDEs**: scaling FBPINNs to higher-dimensional partial differential equations.
- **Dissemination**: presenting these results at appropriate scientific venues.

12 Conclusion

Physics-Informed Neural Networks provide a unified, flexible framework for solving ODEs—both IVPs and BVPs—without labelled solution data. The three IVP constraint strategies (soft, hard, quadrature) offer different trade-offs between generality, stability, and cost. The hybrid Euler–PINN achieves errors of order 10^{-5}, while the PINN inverse-problem formulation recovers unknown parameters to within 0.01% from noisy data.

For BVPs, both soft and hard boundary-condition enforcement were successfully implemented and benchmarked. Spectral bias in standard PINNs motivated the FBPINN architecture, and the subsequent ELM-FBPINN reduces training times by up to four orders of magnitude by converting the optimisation to a linear-algebra problem.

Across all experiments, JAX consistently outperformed PyTorch by 5–7×, underscoring the practical importance of JIT compilation and vectorised automatic differentiation in scientific machine learning.

Acknowledgements

The author gratefully acknowledges the supporting material and inspiration provided by **Prof. Ben Moseley** (University of Oxford) and **Prof. Ovidiu Calin** (Eastern Michigan University). Prof. Moseley's work on Finite Basis PINNs and ELM-FBPINNs formed the foundation for the advanced sections of this report. Prof. Calin's mathematical exposition of neural-network methods for differential equations provided essential background for the theoretical treatment presented here.